

Real-Time Human Tracking with mmWave Radar

Final Report

Timofey Kreslo, Thomas Kemper, Usejd Aliu, Timaël Andrié

June 1, 2025

1 Introduction

This project explores the design and implementation of a real-time system to detect and track people using millimeter-wave (mmWave) radar. We use the TI AWR1843BOOST radar module to create a full processing pipeline — from raw signal capture to multi-target tracking and visualization.

Unlike cameras, this system does not rely on light or images. That means it can still detect people in complete darkness, in smoke, or in other challenging conditions where cameras fail. It can also detect people even when there are small obstacles in the way, like furniture or thin walls.

Since no photos or video are taken, the system is privacy-friendly and well-suited for public spaces. This technology can be used in areas like bus stops, elevators, offices, or other crowded environments where it's important to count or monitor people without using a camera. The final system features:

- Real-time radar signal processing
- Live Statistics (Speed, Location, ...)
- Angular beamforming with Range FFT
- Multi-Target tracking with GTrack algorithm
- People-Counting and Localization in 2D cartesian plane

2 System Overview

The end-to-end radar system consists of eight main modules operating in sequence:

1. ADC Capture — Raw radar signals are collected using the DCA1000EVM interface and AWR1843BOOST radar, forming the basis for processing.
2. Background Removal — An adaptive subtraction method compares each new frame with the previous one to remove static clutter and isolate moving objects.
3. CFAR Detection — A Constant False Alarm Rate (CFAR) algorithm identifies significant reflections in the range-Doppler domain that indicate motion.
4. Beamforming — Simulates antenna steering to compute the angle of arrival (azimuth) for each detection, giving angular localization.
5. Output Normalization — The beamformed heatmap is power-scaled and dynamically normalized to highlight strong detections and suppress noise.

6. Conversion to Detection Points — The beamforming output is converted into a list of position, velocity, and SNR values that serve as inputs for tracking.
7. GTrack Multi-Target Tracking — A Kalman filter-based algorithm that performs prediction, association, and clustering to maintain stable tracks of multiple targets.
8. Visualization & Live Statistics — The output is visualized in real time with object positions, motion trails, presence detection, and statistics like speed or count.

3 Detailed System Description

3.1 ADC Capture

Raw data is captured in real time using the DCA1000 interface connected to the AWR1843BOOST radar module. The radar is configured using LUA scripts — `1843_config.lua`, `1843_record.lua`, `1843_config_streaming.lua`, which define parameters such as waveform setup and frame structure. The radar operates at a center frequency of 77 GHz with a bandwidth of 500 Mhz and uses 3 transmit and 4 receive antennas, resulting in 12 virtual channels. The system is configured with 992 range bins and 16 Doppler bins, enabling the application of range and Doppler FFTs for motion and distance estimation.

Initially, we encountered an issue with the radar as it required the use of TI’s application, *mmWave Studio*, which is exclusively available on Windows. Consequently, some team members using Macs faced significant accessibility challenges. Although virtual machines were explored as a potential solution, they proved ineffective due to complexities in interfacing physical hardware with the VMs.

Following extensive research, we discovered the [mmwave-capture-std](#) project, which provides a comprehensive API for direct communication with the radar hardware. Leveraging the shared UNIX foundation between Linux and macOS, we successfully adapted this solution to macOS.

Previously, our workflow required *mmWave Studio* to trigger the radar and separate Python scripts for data processing. Now, the entire pipeline—from radar triggering to data processing—is consolidated into Python. This integration significantly streamlines our workflow, reduces complexity, and substantially enhances efficiency.

3.2 Background Removal

After reshaping the raw data and applying a Hamming window, we perform an FFT along the range dimension and subtract the previous frame’s FFT output to remove static reflections. This step effectively removes static signals. However, humans, even when appearing stationary, exhibit minor movements, which result in detectable fluctuations and thus maintain a strong signal. Despite this approach, certain static objects, particularly metallic items, still exhibit significant fluctuations, leading to persistent strong signals. To address this issue, the most practical solution we found is manually zeroing the bins corresponding to these undesired signals. While not ideal, as it necessitates manual adjustments each time the scene changes, this method simplifies the process significantly and proves highly effective.

3.3 CFAR Detection

While adapting our project for real-time performance, we faced challenges due to the computational overhead of beamforming across all range bins. Research led us to a solution detailed in [this paper](#). The proposed pipeline utilizes CFAR detection to identify range bins containing potential targets, limiting beamforming only to these specific bins. This optimization significantly increased processing speed and overall system performance.

CFAR is applied on the range-Doppler (RD) cube with the following steps:

1. Compute Doppler FFT over chirps to obtain the RD cube (FFT over range bins is previously completed).

2. Convert the cube’s power data to a magnitude representation.
3. Apply a 2D CA-CFAR algorithm using guard and training cells to detect peaks.
4. Retain only the high-power detections, enhancing the quality of subsequent processing stages.

3.4 Beamforming

As mentioned previously, beamforming is computed exclusively at the CFAR-detected range bins to optimize performance. Initially, we performed beamforming along both horizontal (azimuth) and vertical (elevation) axes. However, since the individuals we aimed to detect remained at a consistent height, we deduced that elevation beamforming was unnecessary. Consequently, we simplified our approach by summing signals vertically. This adjustment significantly improved computational efficiency. The algorithm works as follow:

- Uses the 2D antenna position layout to generate phase shift grids.
- Computes power sums over azimuth directions : $P(\phi, r) = |\sum_{n=0}^{N-1} x_n(r) \cdot e^{-j2\pi \frac{d_n \cos(\phi)}{\lambda}}|$
- This gives fine angular resolution without incurring full-frame computation.

3.5 Normalization

After beamforming, the output is normalized by dividing each value by the maximum and then applying power scaling (e.g., raising to the 8th power) to enhance contrast and highlight strong reflections from humans. This works well for single-person scenes, but in multi-person scenarios, using a high power (like 8) can suppress weaker signals from other targets. Reducing the power (e.g., to 5) helps retain those weaker detections while still providing good visual clarity, making it more suitable for tracking multiple people simultaneously.

3.6 Conversion to Detection Points

With the normalized output, we convert the heatmap points into detection points, which is the required input format for GTrack. Each detection point includes attributes for range, azimuth angle, radial velocity, and signal-to-noise ratio (SNR). To simplify, radial velocities are set to 0, as computing precise velocities for each detection point would significantly increase computational complexity. The SNR values are derived directly from the normalized beamforming output.

3.7 GTrack Module

The final data processing step is the GTrack module. Initially, we considered using only DBSCAN clustering for people counting but quickly identified significant limitations due to high noise levels and signal fluctuations. Thus, we adopted the GTrack algorithm ([link](#)), which seamlessly integrated with our existing pipeline. To implement Gtrack, we rewrite TI’s original C implementation in Python.

The GTrack module executes real-time multi-target tracking through a structured filter-associate-allocate-update-recycle process, including:

1. Boundary Filtering — Discarding points outside the predefined scene.
2. Prediction — Predicting next states of tracks using a Kalman filter.
3. Association — Matching detections to existing tracks based on proximity scoring.
4. Allocation — Using DBSCAN to cluster and identify new tracks.
5. Update — Refining tracks via Kalman updates based on matched detections.
6. Track Maintenance — Removing inactive tracks to conserve resources.
7. Presence Estimation — Flagging tracks that enter specific zones to trigger presence detection and support higher-level decision-making.

While many tracking systems use an Extended Kalman Filter (EKF), our implementation employs a standard linear Kalman filter to maintain computational efficiency without significantly sacrificing accuracy. This choice ensures smooth tracking performance, even on limited hardware.

GTrack outputs a list of active tracks, each containing position, velocity, ID, and confidence. These are visualized in real time, enabling clear identification and counting of multiple people in dynamic scenes.

3.8 Visualization

We visualize real-time beamformed maps and tracked targets using Matplotlib and Panda3D. Each tracked object includes its ID and motion trail, and the interface displays frame rate and presence alerts.

During testing, we observed significant performance issues due to slow plotting. Initially, we cleared and recreated a new visualization window for each frame. To improve efficiency, we modified our approach by only clearing previous drawings and redrawing new data within the existing window. This optimization resulted in substantial performance gains, increasing our frame rate from 7-8 fps to approximately 13-15 fps.

4 Integration and Fusion of Two Radars

Until now, we have described our processing pipeline using only one radar. However, in our proposed solution, we aim to use two radars simultaneously to achieve a more precise heatmap with reduced fluctuations.

To operate a radar, we initiate it by providing a configuration file, after which it begins capturing data. Once running, the radar continuously transmits raw ADC data over an Ethernet connection. To deploy two radars, we need to assign each radar a unique IP address and configure them to transmit data over different ports.

During our tests, we discovered that TI's application, mmWave Studio, does not support simultaneous operation of two radars. Nevertheless, our Mac OS setup successfully handled multiple radars at the same time. The main challenge here involved modifying the radar configuration settings, specifically the IP addresses and port assignments, to ensure distinct communication channels for each radar. We managed to do so by following TI's official documentation.

After establishing simultaneous radar operations, we updated our program. Originally, our program used a single producer process to capture and process data, and a single consumer process to plot results. Transitioning to two radars, we implemented two separate producer processes, each dedicated to handling the complete initial data processing stages (1 to 4) independently. This parallelization significantly enhances processing efficiency by leveraging multi-core CPU architectures.

With both radars actively processing data, the subsequent challenge was combining their respective outputs. To merge the two heatmaps effectively, we first converted each heatmap from polar coordinates to Cartesian coordinates, enabling straightforward spatial alignment and fusion. Given that the radars are positioned side-by-side and thus not identically located, we introduced a spatial offset to align the heatmaps accurately. After alignment, we multiplied the heatmaps element-wise to fuse the data, enhancing detection precision by reinforcing common detections and suppressing random noise.

To maintain compatibility with our existing tracking module, GTrack, we converted the combined Cartesian heatmap back into polar coordinates. This approach avoided substantial modifications to GTrack, which would have required considerable development effort.

Finally, the fused heatmap undergoes the remaining processing stages (steps 5 through 8), concluding our pipeline and enabling robust tracking and detection of individuals.

5 Results

Our system demonstrates robust real-time performance and pretty accurate human tracking capabilities across a variety of challenging scenarios:

- **Real-Time Performance:** The system consistently operates at approximately 15 frames per second, enabling smooth and responsive visualization of people’s movements in real time.
- **Accurate 2D Localization:** By combining range and angular information through beamforming, we achieve precise 2D localization. This is visualized in both polar projection (range vs. angle) and Cartesian coordinates (x, y), providing a complete spatial understanding of the scene.
- **Multi-Target Tracking with Unique IDs:** The GTrack module assigns a unique identifier to each person, maintaining their identity across frames even as they move. This ensures continuous tracking of multiple people simultaneously, without track switching or loss.
- **Motion Visualization:** To convey direction and speed intuitively, we overlay arrows on each tracked individual. The arrow length dynamically scales with the person’s speed, pointing in the direction of movement, effectively visualizing both velocity magnitude and orientation.
- **Robustness Through Occlusions:** Our system handles partial occlusions well, such as when people walk behind one another or near static objects. The tracking remains stable thanks to Kalman filtering and intelligent data association.
- **Clutter Resilience:** Adaptive background subtraction and SNR filtering help suppress static noise and reduce false detections, allowing the system to recover and maintain tracks even in complex scenes.
- **Through-Wall Detection:** One of the most significant achievements was the system’s ability to detect and track individuals behind a thin wall made of carton. This demonstrates the core advantage of mmWave radar — penetrating materials that block light — and highlights the system’s potential in real-world, obstructed environments.

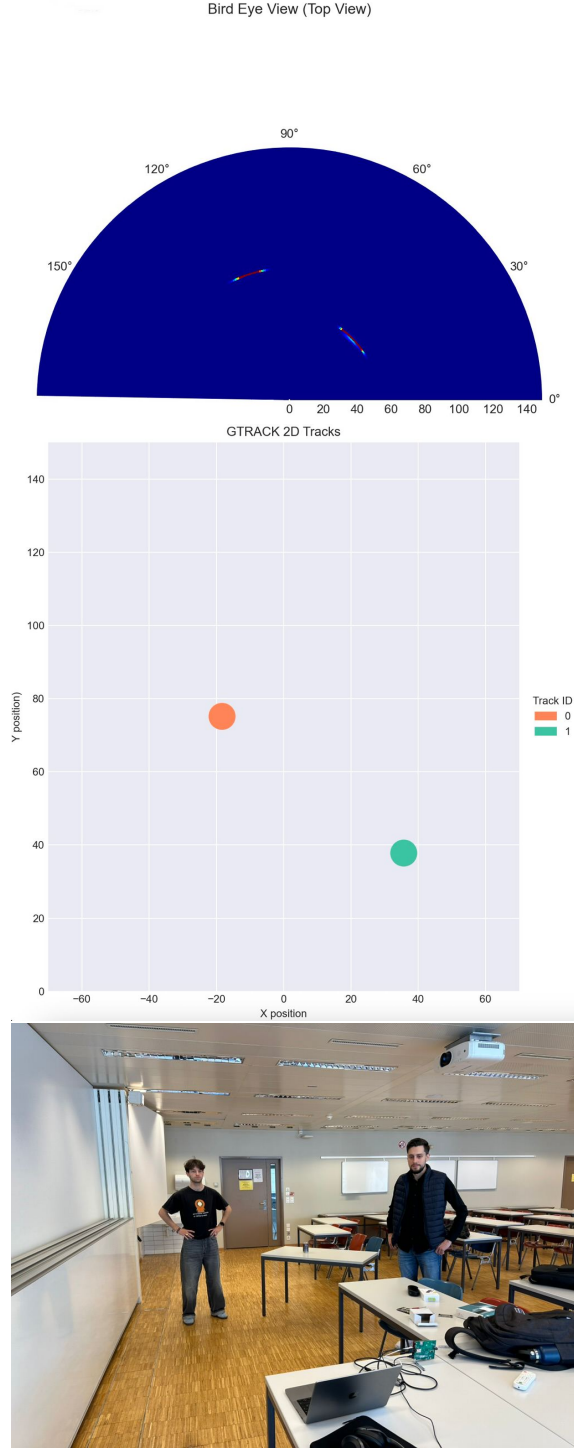


Figure 1: Real-time visualization showing tracked positions on polar and cartesian 2D projections after GTrack, and the corresponding real-scene with 2 persons in a classroom with obstacles.

5.1 Recordings

[Google Drive with Videos of our results](#)

In Final Results folder: to help interpret the videos, we followed a simple naming convention:

"2r" — Two radars used for the test

"1p" / "2p" / "4p" — Number of people in the scene (1 person or 2 people or 4 people)

"1", "2" — Take number or repetition of the test

"c" — Indicates a carton wall was placed in front of the radar to simulate through-obstacle detection

"l" — Indicates a long-range scenario was tested

Examples:

2r_1p_1 → Test using 2 radars, 1 person, take 1

2r_2p_c → Test using 2 radars, 2 people, with carton obstacle

2r_1p_l → Test using 2 radars, 1 person, long-range test

6 Issues & Challenges

We encountered several important challenges throughout the development and testing of our system:

- **Radar Configuration Limitations TI's** — TI's mmWave Studio does not support running two radars simultaneously. To overcome this, we moved away from TI's software stack and used Python to configure and stream data from both radars. We manually adjusted IP addresses and ports to establish distinct communication channels, achieving full dual-radar compatibility on macOS.
- **Beamforming Normalization Challenges** — When tracking multiple people, normalization by the strongest signal could suppress weaker reflections. If one person's radar return was significantly stronger, others could become nearly invisible. To prevent this, we reduced power scaling from 8 to 5 during multi-person tracking, improving visibility while preserving signal contrast.
- **Dual-Radar Fusion Effects** — Combining outputs from two radars via element-wise multiplication acts like a logical AND, preserving only strong overlapping detections. While this improves precision and noise suppression, it can also eliminate valid detections captured by only one radar, leading to missed targets in some configurations.
- **Track ID Swapping** — If two people stood at similar ranges and one produced a much stronger reflection, the other's signal could be severely attenuated by normalization. This often caused temporary disappearance. When the person reappeared, GTrack sometimes reassigned them a new ID or incorrectly swapped their identity with another. We considered using a buffer of several past frames to better stabilize identity assignment but were unable to implement it in time.
- **Occlusion Handling** — When one person passed directly in front of another, the radar signal was blocked, causing the person behind to disappear momentarily. Placing a second radar perpendicular to the first could alleviate this by capturing motion from another perspective, but we instead positioned the radars in parallel, which still improved angular accuracy.
- **Kalman Filter Over-Prediction** — The Kalman filter occasionally predicted motion for a stationary target, causing the display to jitter. These oscillations occur because the filter continues forecasting based on prior velocity. This could be mitigated by more advanced tuning or using adaptive models, but we opted for a simpler implementation to preserve real-time performance.
- **OS Compatibility** — TI's ecosystem is tightly coupled to Windows, making it difficult to develop on other platforms. However, we successfully built a fully Python-based configuration and streaming pipeline that ran entirely on macOS, including support for dual radars, by bypassing mmWave Studio and directly using low-level commands.

7 Code & Ressources

- [Our Github Repository](#)
- Core Code: prod.dca.py, module.py, utils.py, realtime_streaming.py
- Radar Configs: 1843_config.lua, 1843_record.lua, 1843_config_streaming.lua
- Library Dependencies: NumPy, SciPy, scikit-learn, Matplotlib, Panda3D
- [TI Developer Zone - Tracking with GTRACK](#)
- [People Counting Using Low-Cost FMCW MIMO Radar](#)

```
def producer_real_time_1843(q, cfg_radar, cfg_gtrack, cfg_cfar, gtrack):
    while True:
        # Read data from DCA1000
        raw = dca.read(timeout=0.5, chirps=chirp_loops...)

        # Apply Hamming window
        adc_windowed = raw * np.hamming(adc_samples)

        # Apply FFT along the range dimension
        range_fft = np.fft.fft(adc_windowed, axis=-1)
        last_frame_fft = np.fft.fft(last_frame, axis=-1)

        # Subtract the last fram and 0 the static clutter
        range_fft_s = range_fft - last_frame_fft
        range_fft_s[:, :, 0:20] = 0
        range_fft_s[:, :, 120:150] = 0
        range_fft_s = range_fft_s[:, :, r_idxxs]
        last_frame = beat_freq_data

        # Compute CFAR
        dets = process_frame(range_fft_s, cfg_cfar)

        # Compute beamforming
        bf_output, detection = beamform_2d_s(range_fft_s, cfg_radar, x_locs
           [:,0], dets)

        # Compute the SNR
        snrs = np.array([d.snr for d in detection])
        snrs_max = np.max(snrs)
        detection_tuned = []
        for d in detection:
            d_t = d
            d_t.snr = d_t.snr / snrs_max
            detection_tuned.append(d_t)

        # Keep only the strong detections
        detection = [d for d in detection_tuned if d.snr >= cfg_gtrack.
            min_snr_threshold]

        # Normalize the output
        bf_output = np.abs(bf_output)
        to_plot = bf_output
        to_plot /= np.max(to_plot)
        to_plot = to_plot ** 8

        # Compute GTRACK
        gtrack_output = tracker.step(detection)
```

Listing 1: Real-Time Radar Producer Pipeline (Python)

8 Conclusion

This project successfully implemented a real-time, radar-based system for detecting, localizing, and tracking multiple people using mmWave technology. Through a carefully structured processing pipeline — including background subtraction, CFAR detection, angular beamforming, and multi-target tracking with GTrack — we demonstrated the system’s ability to operate reliably even in challenging environments such as darkness, smoke, or areas with small obstacles.

One of the core strengths of our approach is its privacy-preserving design, which avoids cameras entirely and enables safe deployment in sensitive or crowded environments like elevators, public transit hubs, or workplaces. Additionally, our use of a modular architecture ensures that each component — from signal capture to visualization — can be independently improved, extended, or replaced to suit future applications.

The successful reimplementaion of GTrack in Python further showcases the system’s flexibility and adaptability to different hardware and software ecosystems. With accurate real-time tracking, presence detection, and live visual feedback, the system lays the groundwork for future applications in smart infrastructure, security, and human-aware automation.